

RCP103 – Rapport de TP

TP02 - Création d'un simulateur à évènements discrets

BOURDAYA Amina, BOUTY Gabriel, CLERC Julien

18 mai 2026

Résumé

Ce TP a pour objectif de créer un programme permettant de simuler un système simple de type Client(s)/Serveur asynchrones. Ce simulateur nous permettra de réaliser des mesures variées pour qualifier notre système (nombre instantané de requêtes, temps d'attente, etc.), sans avoir à mettre en œuvre concrètement ce dernier.

1 Description du système simulé

Comme évoqué en introduction, ce rapport présente la réalisation d'un simple simulateur à évènements discrets du type Client(s)/Serveur :

Un ensemble de N clients ($N \geq 1$), envoient des messages à 1 serveur. Un portail sert d'intermédiaire entre les clients et le serveur. Compte tenu que le serveur ne peut traiter qu'un message à la fois, le portail permet de temporiser les messages en attente de traitement dans une file de taille fixe K .

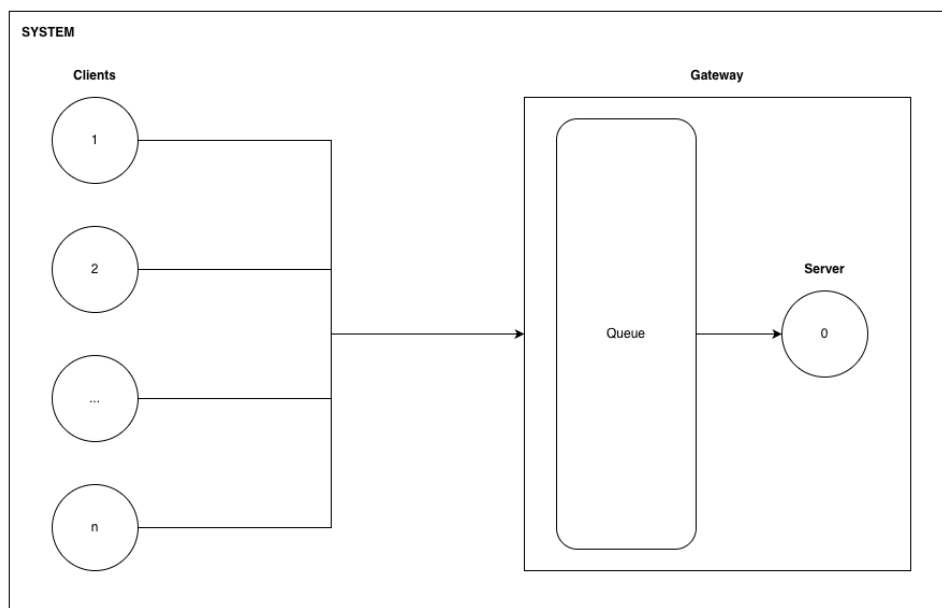


FIGURE 1 – Schéma du système simulé

Dans ce simulateur, les envois de messages suivent un processus de Poisson : le taux d'envois est constant dans le temps, les temps entre deux envois consécutifs suivent une distribution exponentielle. De même, le temps de service suit une loi exponentielle.

Compte tenu du fait que nous n'avons pour l'instant qu'un seul serveur, nous pouvons donc définir cette simulation comme un processus de file d'attente de type $M/M/1/K$:

- M : Arrivées selon un processus de Poisson (Markovien).
- M : Temps de service selon une distribution exponentielle (Markovien).
- 1 : Nombre de serveurs (ici, 1).
- K : Capacité du système (ici, la taille de la file du portail de capacité K).

La configuration du simulateur est définie comme suit :

- Temps moyen inter-message : configurable
- Temps de transmission : $1t$
- Temps moyen de service : configurable
- Temps total de la simulation : configurable

2 Architecture du simulateur

2.1 Architecture initiale

Afin de simuler les transmissions et traitement de messages entre le ou les *Client* et le *Server* (à travers le *Gateway*), chaque étape de gestion du *Message* sera encapsulée dans des *Event*. Ces *Event* pourront être, de fait, de 3 types différents :

- `SEND_MSG` : Le message est envoyé par le client.
- `RECV_MSG` : Le message est reçu par le portail.
- `MSG_DPT` : Le message est transmis au serveur (si ce dernier est disponible).

Chacun de ces *Event* sera timestampé et stocké chronologiquement dans un ordonnanceur (*Scheduler*). Ainsi, en ajoutant progressivement des événements à notre *Scheduler*, et en itérant sur ces derniers, nous pourrions simuler l'écoulement du temps, en nous concentrant uniquement sur les échantillons temporels qui nous intéressent.

L'ensemble de ces composants seront encapsulés dans un *Engine*, dont la boucle principale viendra interagir avec ces derniers afin de dérouler la simulation.

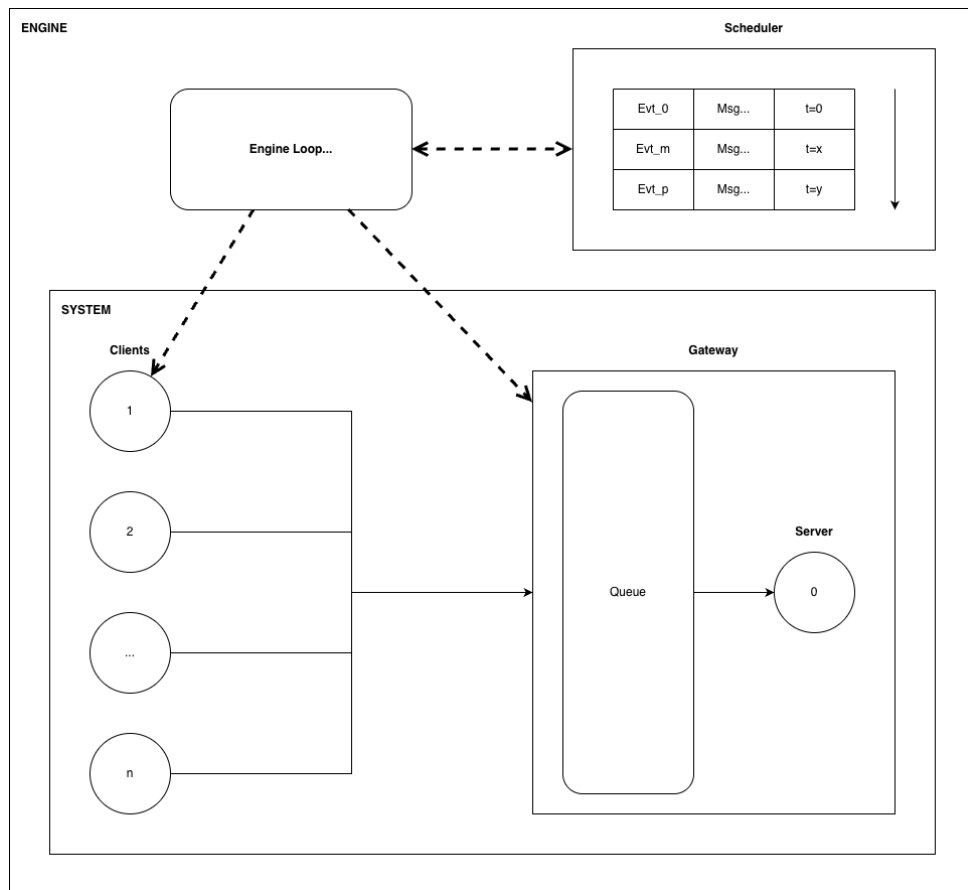


FIGURE 2 – Schéma de principe du simulateur

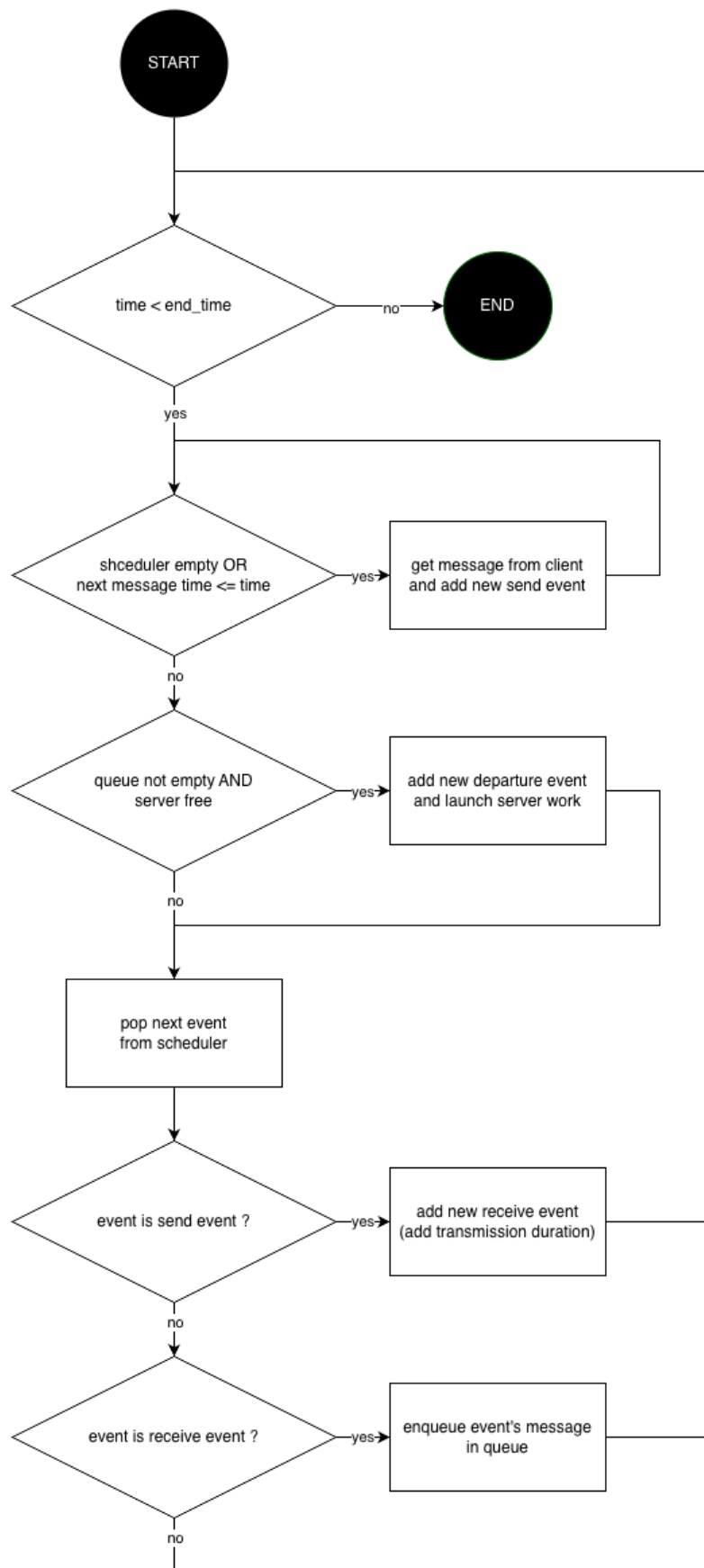


FIGURE 3 – Algorithme de la boucle principale de l'Engine

2.2 Organisation du code source

Le dossier fourni s'organise ainsi :

- **root** : Le point d'entrée du programme principal (`tp02_main.py`), les fichiers de configurations de lancement des tests, ainsi que les dépendances Python.
- Dossier **sources** : Le code applicatif.
- Dossier **tests** : Le code relatif aux tests unitaires.
- Dossiers **report** et **images** : Le présent rapport et les images associées.

3 Création des premières classes

3.1 Les données : class Message et class Event

Comme évoqué précédemment, la classe *Message* est l'unité d'échange entre *Client* et *Server*. Cette classe contient :

- L'identifiant du message : `message_id`
- L'identifiant du nœud émetteur : `source`
- L'identifiant du nœud destination : `destination`
- L'instant auquel le message est envoyé par le client : `send_time`
- L'instant auquel le message arrive à la passerelle : `arrival_time`
- L'instant auquel le traitement du message commence par le serveur : `server_start`

Ce *Message* est encapsulé dans un *Event*, dont les attributs sont les suivants :

- L'identifiant de l'évènement : `id`
- Le type de l'évènement : `type`
- Le *Message* encapsulé : `message`

Event possède par ailleurs un accesseur permettant de déterminer son timestamp, en fonction de son type.

3.2 L'ordonnanceur : class Scheduler

La classe *Scheduler* est chargée de gérer les futurs évènements du simulateur. Elle contient une liste d'évènements triés par ordre chronologique.

Lorsqu'un nouvel évènement est ajouté, la méthode parcourt la liste depuis la fin afin de trouver la bonne position d'insertion. Tant que le temps de l'évènement déjà présent est supérieur au temps du nouvel évènement, on remonte la liste. Lorsque la bonne position est trouvée, le nouvel évènement est inséré avec `insert`. Cela permet de garder la liste triée sans utiliser `sort` après chaque ajout.

La méthode `pop_event` retire et retourne le premier évènement de la liste. Comme la liste est maintenue triée par temps d'exécution, cet évènement correspond toujours au prochain évènement à traiter.

La méthode `get_current_time` retourne le temps du premier évènement de la liste, c'est-à-dire le temps courant du simulateur. Le *Scheduler* possède une mémoire du timestamp du dernier évènement retiré, afin de fournir toujours un temps de référence, même si sa liste d'évènement est vide.

La méthode `has_events` permet de savoir s'il reste encore des évènements à traiter.

Enfin, le *Scheduler* enregistre progressivement les évènements retirés, afin de pouvoir garder une trace de ces derniers.

3.3 Les composants simulés : class Client et Server

Les classes *Client* et *Server* représentent les composants principaux du système simulé. *Client* émet les messages qui seront traités par *Server*.

Comme décrit précédemment, les temps d'émissions comme les temps de traitement sont définis par une distribution exponentielle. Ainsi, on pourra définir pour chaque instance de *Client* et *Server*, une moyenne : pour *Client*, cette valeur correspond au nombre moyen de message émis par unité de temps, et pour *Server*, cette valeur correspond au nombre moyen de message traités par unité de temps.

Client expose quelques fonctions minimales :

- `get_next_msg_time` : permettant de connaître le timestamp du prochain envoi de *Message* émis par le *Client*.
- `pop_message` : permettant de simuler l'envoi du *Message*. Une instance préconfigurée de *Message* est renvoyée, et le timestamp interne est incrémenté d'une valeur aléatoire, générée selon une distribution exponentielle.

Server expose aussi quelques fonctions minimales :

- `is_free` : permettant de savoir si le *Server* est actuellement en train de traiter virtuellement un *Message*.
- `start_work` : permettant de simuler le début du traitement d'un *Message*. Un timestamp interne de fin de travail est défini selon le timestamp passé en argument, augmenté d'une valeur aléatoire, générée là aussi selon une distribution exponentielle.

3.4 Les fonctions de logs : fonctions Trace

Le module `trace` contient trois fonctions fournissant un système de journalisation au simulateur, permettant de visualiser et d'enregistrer les événements.

Voici le prototype et la description de chaque fonction :

`def get_time_and_node(e: Event)` : prend un événement en paramètre et retourne un tuple constitué du numéro de nœud et du temps en fonction du type d'événement.

`def generateTraceOut(t_event: list[Event])` : prend une liste d'événements en paramètre et écrit sur la sortie standard l'ensemble des événements contenu dans la liste.

`def generateTraceCSV(t_event: list[Event])` : prend une liste d'événements en paramètre et écrit dans un fichier nommé `trace.csv` l'ensemble des événements contenu dans la liste au format CSV. Le délimiteur utilisé est le caractère `','`.

3.5 Le cœur du système : class Engine

La classe *Engine* représente le cœur du simulateur. Elle définit les instances des différents composants (*Client*, *Server*, etc.) lors de son initialisation, et simule les transactions entre chacun d'entre eux durant l'exécution de la simulation.

Sa fonction `run` permet de lancer l'exécution. Celle-ci s'appuie sur un *Scheduler* pour déterminer l'évolution du temps. Son rôle principale est de forcer les comportements de chacun des composants (*Client*, *Server*, *Queue*, etc.) et d'encapsuler les *Message* émis, reçu, traités dans des *Event*, afin d'alimenter le *Scheduler*.

La classe *Engine* propose aussi une fonction `log` permettant d'afficher les *Event* déjà passés, grâce aux fonctions proposées par le module `trace`.

3.6 Le point d'entrée : fonction main

Le point d'entrée du programme se trouve dans `tp02_main.py`. Il permet d'exécuter le programme simplement en appelant `python tp02_main.py` depuis le répertoire racine.

En l'état, il se contente de créer une instance d'*Engine*, et de lancer l'exécution de cette instance. Mais on pourrait par exemple imaginer y ajouter un parsing d'arguments pour configurer le simulateur dynamiquement.

3.7 La file d'attente des événements : class Queue

La classe `Queue` est une structure de donnée de type FIFO (First In First Out). Elle va garder les messages dans l'attente de leur prise en compte par le serveur.

Voici le prototype et la description de chaque méthodes :

```
def put(self, msg: Message) : ajoute un message dans la file.
```

```
def get(self) : récupère le message le plus ancien ajouté dans la file.
```

```
def is_empty(self) : indique si la file est vide.
```

```
def size(self) : renvoie la taille de la file c'est-à-dire le nombre de message.
```

```
def index(self, msg: Message) : renvoie l'index d'un message dans la file. Cette méthode est utile pour les tests.
```

4 Tests des premières classes

4.1 Le système de test : Pytest

Afin de vérifier le bon fonctionnement des classes implémentées, des tests unitaires ont été réalisés avec l'aide de l'utilitaire `Pytest`. L'objectif est de tester chaque classe séparément avant de l'intégrer dans le simulateur. L'ensemble des tests sont regroupés dans le répertoire `tests`.

4.2 Présentation des tests rédigés

Test de *Message* :

- Le test `test_message` vérifie que le constructeur initialise correctement les attributs du message (l'identifiant, la source et la destination, les différents timestamp).
- Le test `test_message_setters` vérifie que les setters modifient correctement les valeurs internes de l'objet, après modification de l'identifiant, de la source, de la destination, et des différents temps, les getters sont utilisés pour vérifier que les nouvelles valeurs ont bien été enregistrées.
- Le test `test_message_str` vérifie la méthode d'affichage de la classe message. Cette méthode est utile pour le débogage, car elle permet d'afficher rapidement les informations principales d'un message.

Test de *Event* :

- Le test `test_event_init` vérifie que le constructeur initialise correctement le temps de l'événement et son type. Un événement de type `SEND_MSG` est créé avec un temps égal à 1.5, puis les getters permettent de vérifier que ces valeurs sont bien stockées.

- Le test `test_event_setters` vérifie que le temps et le type d'un événement peuvent être modifiés correctement. Par exemple, un événement initialement de type `SEND_MSG` est modifié en `RECV_MSG`, et son temps est changé de 5.5 à 8.0.
- Le test `test_event_message_content` vérifie que l'événement conserve bien le message qui lui est associé. Après avoir récupéré le message depuis l'événement, le test vérifie que son identifiant, sa source et sa destination sont corrects.
- Le test `test_event_str` vérifie la méthode d'affichage de la classe *Event*. Cette méthode affiche l'identifiant de l'événement, son type, son temps et le message associé. Elle permet donc de faciliter le suivi des événements.

Test de *Server* :

- Le test `test_server` vérifie que la classe *Server* remplit bien son rôle de simulation de serveur, à savoir pouvoir démarrer un job, rester occupé pendant une période de temps aléatoire, puis être à nouveau libre à la fin de cette période de temps.

Test de *Client* :

- Le test `test_client_init` vérifie l'état initial d'un *Client*.
- Les tests `test_get_next_msg_time` et `test_pop_message` vérifient que *Client* renvoie bien un temps qui s'incrémente au fur et à mesure que les *Message* sont virtuellement émis par le *Client*.

Test de *Scheduler* :

- Le test `test_scheduler` vérifie que les temps des événements récupérés sont toujours dans l'ordre chronologique croissant. Cela permet de valider le rôle principal du *Scheduler* : maintenir les événements futurs triés par temps d'exécution. Le test vérifie aussi que les événements passés sont bien enregistrés dans la liste des événements passés.

Test de *Trace* :

- Le test définit par `test_get_time_and_node_event_SEND_MSG` vérifie que la fonction `get_time_and_node_event` renvoie bien le bon tuple en fonction du type de message.
- Le test définit par `test_get_time_and_node_event_RECV_MSG` vérifie que la fonction `get_time_and_node_event` renvoie bien le bon tuple en fonction du type de message.
- Le test définit par `test_get_time_and_node_event_MSG_DEPT` vérifie que la fonction `get_time_and_node_event` renvoie bien le bon tuple en fonction du type de message.
- Le test définit par `test_generateTraceOut_SEND_MSG(capsys)` vérifie que la fonction `generateTraceOut` affiche les bonnes informations en fonction du type de message.
- Le test définit par `test_generateTraceOut_RECV_MSG(capsys)` vérifie que la fonction `generateTraceOut` affiche les bonnes informations en fonction du type de message.
- Le test définit par `test_generateTraceOut_MSG_DEPT(capsys)` vérifie que la fonction `generateTraceOut` affiche les bonnes informations en fonction du type de message.
- Le test définit par `test_generateTraceCSV` vérifie que la fonction `generateTraceCSV` crée le fichier "trace.csv" avec les bonnes valeurs pour plusieurs messages.

Test de *Engine* :

- Le test `test_engine_complete` vérifie que l'exécution du `run` de *Engine* ne tombe pas dans une boucle sans fin. Il vérifie ensuite que les *Event* reçus suivent bien une logique temporelle incrémentale, et qu'ils soient bien cohérents les uns avec les autres. Par exemple, un `MSG_DEPT` doit avoir un `RECV_MSG`, ainsi qu'un `SEND_MSG` correspondants avec des timestamps tel que $t_{\text{MSG_DEPT}} \geq t_{\text{RECV_MSG}} \geq t_{\text{SEND_MSG}}$.

Test de *Queue* :

- Le test définit par `test_put` vérifie que la méthode `put` ajoute correctement un message.

- Le test défini par `test_get` vérifie que la méthode `get` renvoie correctement un message et le supprime de la file d'attente.
- Le test défini par `test_is_empty` vérifie que la méthode `is_empty` renvoie le booléen `True` si la file est vide
- Le test défini par `test_size` vérifie que la méthode `size` renvoie la taille de la file.
- Le test défini par `test_str` vérifie que la méthode `__str__` renvoie la chaîne de caractère comprenant l'ensemble des éléments de la file d'attente.

4.3 Exécution des tests

Les tests peuvent être lancés depuis le répertoire principal en tapant la commande `pytest` (attention, `Pytest` doit être préalablement installé). L'ensemble des tests présents dans le répertoire `tests` seront exécutés.

5 Conclusion

Ce TP nous permet d'appréhender ce que peut être un simulateur à événements discrets. Nous mettons en œuvre des concepts vus en cours tels que les files d'attente ou encore les processus de Poisson.

Il nous permet aussi de comprendre l'intérêt que peut représenter la complémentarité qu'il peut exister entre modélisation mathématique et simulation. En effet, la modélisation mathématique nous fournit les outils (variables aléatoires, processus stochastiques, etc.) qui vont nous permettre de construire facilement un simulateur, et ainsi réaliser, tout aussi facilement, des mesures théoriques sur un système complexe. Établir et mettre en œuvre un protocole de mesures sur ce même système dans le monde réel aurait été bien plus lourd et coûteux.

D'un point de vue du développement logiciel, les différentes itérations de réalisation, basées sur une approche TDD (Test Driven Development), nous permettent de valider progressivement nos composants logiciels. Cela facilite ensuite l'assemblage, et permet aussi d'éviter les régressions.